Technische Hochschule Ingolstadt
Studiengang: Data Science in Technik und Wirtschaft

Kundenakquise durch Data Science

Binäre Klassifikation zur Vorhersage von Termineinlagen

Seminararbeit Machine Learning Lab

Autor:	Hasan Kagan Kurt
Matrikelnummer:	00145290
Abgabedatum:	21.10.2025

Abstract

Die vorliegende Arbeit analysiert den Bank-Marketing-Datensatz, der die Ergebnisse von Telefonkampagnen einer portugiesischen Bank zur Akquise von Termineinlagen enthält. Das Ziel ist die Entwicklung eines Klassifikationsmodells, das vorhersagt, ob ein Kunde das Produkt abonniert (Zielvariable 'y'). Dies dient zur Optimierung der Ressourcen und zur Effizienzsteigerung zukünftiger Marketingaktivitäten.
Auf Basis soziodemografischer, ökonomischer und kampagnenspezifischer Merkmale wurden drei Modelle – die **Logistische Regression** (Baseline), der **Random Forest** und eine **Gradient-Boosting-Maschine (XGBoost)** – verglichen. Der Fokus der Evaluierung lag aufgrund des vorliegenden Klassen-Ungleichgewichts auf dem **ROC AUC Score**. Es zeigte sich, dass die Ensemble-Methoden, insbesondere der XGBoost, die höchste Vorhersageleistung erzielten, während die Logistische Regression wichtige Einblicke in die linearen Zusammenhänge lieferte. Die Ergebnisse verdeutlichen, welche Kundenmerkmale die höchste Wahrscheinlichkeit für einen erfolgreichen Abschluss aufweisen und demonstrieren das Potenzial von Data-Science-Methoden zur datenbasierten Steuerung von Kundenakquise-Strategien.

1 Einleitung

Die effiziente Kundenakquise stellt für Finanzinstitute weltweit eine strategische Herausforderung dar. Insbesondere Telefonmarketing-Kampagnen zur Bewerbung von Sparprodukten, wie Termineinlagen (Termin-Deposits), sind oft mit hohen operativen Kosten verbunden und weisen niedrige Erfolgsquoten auf. Dies erfordert eine präzisere Zielgruppenansprache, um die Ressourcen optimal einzusetzen und die Akzeptanz bei den Kunden zu erhöhen.

Der vorliegende Datensatz stammt von einer Direktmarketing-Kampagne einer portugiesischen Bank und enthält detaillierte Informationen über Kundenmerkmale, Kontakthistorie sowie wirtschaftliche und soziale Kontextvariablen (Moro et al., 2014). Die Zielvariable **'y'** gibt an, ob der jeweilige Kunde nach der letzten Kontaktaufnahme ein Abonnement für eine Termineinlage abgeschlossen hat ('yes') oder nicht ('no'). Damit bietet der Datensatz eine exzellente Grundlage für ein **Binäres Klassifikationsproblem**. Die Entwicklung eines robusten Vorhersagemodells kann der Bank helfen, ihre Marketingstrategie zu optimieren und die Konversionsrate signifikant zu steigern.

Diese Arbeit zielt darauf ab, die Eignung verschiedener Machine-Learning-Modelle für dieses Klassifikationsproblem zu bewerten und die wichtigsten Einflussfaktoren für einen erfolgreichen Vertragsabschluss zu identifizieren. Dazu werden drei etablierte Verfahren des maschinellen Lernens – die **Logistische Regression**, der **Random Forest** und ein **XGBoost** – implementiert und verglichen. Neben der reinen Vorhersagegenauigkeit steht die Fähigkeit der Modelle, komplexe, nicht-lineare Muster zu erkennen, sowie deren Interpretierbarkeit im Fokus. Ziel ist es, eine datengesteuerte Empfehlung für die zukünftige Gestaltung von Telefonmarketing-Kampagnen abzuleiten.

2 Datenanalyse und Vorbereitung

In dieser Phase werden die Rohdaten geladen, bereinigt und für die weitere Verarbeitung strukturiert.

2.1 Laden der Daten und initiale Inspektion

Der verwendete Datensatz wurde aus dem UCI Machine Learning Repository bezogen (Moro et al., 2014). Er liegt im CSV-Format vor und enthält 41.188 Datensätze.

Das Laden der Daten erfolgte mithilfe der Python-Bibliothek *Pandas*. Da die vorliegende Datei (bank-additional-full.csv) Semikolons als Trennzeichen verwendet, musste der Separator-Parameter beim Einlesen explizit spezifiziert werden:

```
df = pd.read_csv('bank-additional-full.csv', sep=";")
```

Eine erste Inspektion ('df.info()') bestätigte, dass der Datensatz 21 Spalten umfasst, die sich in numerische (z.B. *age*, *euribor3m*) und kategoriale Merkmale (z.B. *job*, *education*) unterteilen.

2.2 Bereinigung unrealistischer Features

Eine Analyse der Merkmale zeigte, dass die Spalte `duration` (Gesprächsdauer des letzten Kontakts) einen extrem starken Einfluss auf die Zielvariable hat. Da die Dauer eines Anrufs jedoch erst *nach* dessen Beendigung bekannt ist, steht diese Information zum Zeitpunkt der Vorhersage (d.h. vor dem Anruf) in einem realen Anwendungsszenario nicht zur Verfügung.

Die Einbeziehung dieses Merkmals würde zu einem sogenannten *Data Leakage* führen: Das Modell würde lernen, dass lange Gespräche zu einem Abschluss führen, was zwar statistisch korrekt, aber für die Selektion potenzieller Kunden im Vorfeld nutzlos ist. Aus diesem Grund wurde das Merkmal konsequent aus dem Datensatz entfernt:

```
df = df.drop('duration', axis=1)
```

Durch diesen Schritt wird sichergestellt, dass das Modell nur auf Informationen basiert, die der Bank vor der Kontaktaufnahme tatsächlich vorliegen.

2.3 Kodierung der Zielvariable

Die Zielvariable **'y'** liegt im Datensatz ursprünglich als kategoriales Merkmal mit den Ausprägungen 'yes' und 'no' vor. Für die Verarbeitung durch Machine-Learning-Algorithmen ist eine numerische Repräsentation erforderlich. Daher wurde eine binäre Kodierung vorgenommen, bei der 'yes' auf 1 (Abonnement abgeschlossen) und 'no' auf 0 (kein Abonnement) abgebildet wird:

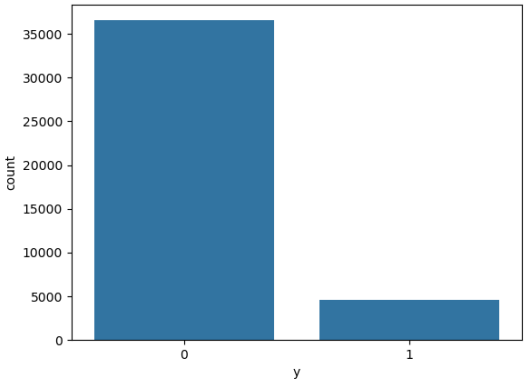
```
df['y'] = df['y'].map({'yes': 1, 'no': 0})
```

Dies ermöglicht es den Klassifikationsmodellen, die Wahrscheinlichkeit für die Klasse 1 (positiver Abschluss) direkt zu schätzen.

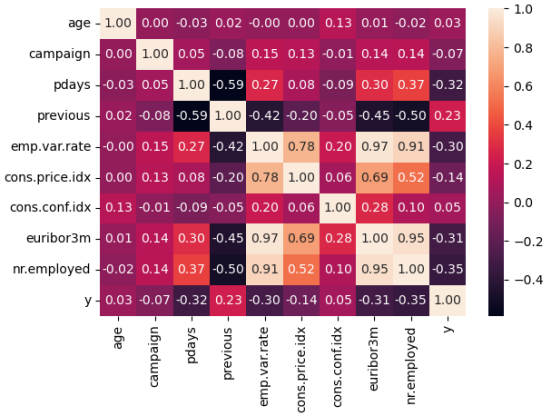
2.4 Analyse des Klassen-Ungleichgewichts und der Korrelationen

Die Analyse der Zielvariablen zeigt zunächst ein deutliches Ungleichgewicht zwischen den Klassen (Class Imbalance). Zusätzlich wurden die linearen Abhängigkeiten zwischen den numerischen Merkmalen untersucht.

Die folgende Abbildung 1 fasst beide Aspekte zusammen: Links ist die ungleiche Verteilung der Zielvariablen zu sehen, rechts die Korrelationsmatrix der numerischen Features.



(a) Verteilung der Zielvariablen (ca. 11 % 'yes')



(b) Korrelationsmatrix der Features

Abbildung 1: Datenanalyse: Klassenverteilung (links) und Korrelationen (rechts). Während das Klassenverhältnis stark ungleich ist, zeigen sich in der Matrix (rechts) starke Zusammenhänge zwischen makroökonomischen Indikatoren.

2.5 Feature Selection und Behandlung von Multikollinearität

Nach der ersten Analyse der Korrelationsmatrix (siehe Abbildung 1b) wurde eine gezielte Auswahl der Merkmale vorgenommen. Dabei wurden die Features `emp.var.rate` und `euribor3m` aus dem Datensatz entfernt.

Die Entscheidung basiert auf folgenden Überlegungen:

1. **Multikollinearität:** Die makroökonomischen Indikatoren im Datensatz zeigen untereinander extrem hohe Korrelationskoeffizienten (oft $> 0,9$). Wenn Merkmale fast identische Informationen liefern, kann dies insbesondere bei der Logistischen Regression zu instabilen Modellkoeffizienten führen.

2. **Modellkomplexität und Redundanz:** Da der Indikator `nr.employed` (Anzahl der Beschäftigten) bereits eine ähnliche wirtschaftliche Dynamik abbildet, liefern `euribor3m` und `emp.var.rate` kaum zusätzliche erklärende Varianz. Durch deren Entfernung wird das Modell schlanker und weniger anfällig für Overfitting, ohne signifikant an Vorhersagekraft zu verlieren.

Durch diese gezielte Reduktion wird sichergestellt, dass das Modell robuster gegenüber Schwankungen in einzelnen, hochkorrelierten Wirtschaftsdaten wird.

2.6 Aufteilung in Trainings- und Testdaten

Um die Generalisierungsfähigkeit der Modelle objektiv zu bewerten und Overfitting zu vermeiden, wurde der Datensatz vor der weiteren Verarbeitung in Trainings- und Testdaten aufgeteilt. Es wurde ein Split-Verhältnis von 80 % zu 20 % gewählt.

Durch die Aufteilung vor der Anwendung des `StandardScaler` wird sichergestellt, dass statistische Informationen (wie Mittelwert und Standardabweichung) ausschließlich aus den Trainingsdaten gewonnen werden. Dies verhindert ein sogenanntes *Data Leakage*, bei dem Informationen aus dem Testset bereits in den Trainingsprozess einfließen würden.

```
from sklearn.model_selection import train_test_split

X = df.drop('y', axis=1)
y = df['y']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

Dabei wurde der Parameter `stratify=y` verwendet, um das in Abschnitt 2.4 beschriebene Klassen-Ungleichgewicht in beiden Teildatensätzen proportional beizubehalten.

2.7 Definition des Präprozessors

Um die unterschiedlichen Datentypen gleichwertig zu behandeln, wird ein zentraler `preprocessor` definiert. Technisch handelt es sich dabei um einen `ColumnTransformer`, der Transformationen parallel auf spezifische Spalten anwendet und die Ergebnisse anschließend zusammensetzt.

Der Prozess umfasst zwei Hauptkomponenten:

1. **Numerische Transformation:** Für numerische Merkmale wird der `StandardScaler` angewendet, um die Werte zu standardisieren.

2. **Kategoriale Transformation:** Kategoriale Merkmale werden mittels `OneHotEncoder` in binäre Vektoren überführt.

Der `preprocessor` wendet diese Schritte auf die definierten Spaltenlisten (`numerical_features` und `categorical_features`) an. Spalten, die nicht explizit genannt werden, bleiben durch die Einstellung `remainder='passthrough'` erhalten.

Der folgende Code zeigt die Implementierung des Präprozessors:

```
numerical_transformer = Pipeline(steps=[
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numerical_transformer, numerical_features),
        ('cat', categorical_transformer, categorical_features)
    ],
    remainder='passthrough'
)
```

3 Modelltraining, Evaluierung und Interpretation

In diesem Abschnitt werden die Modelle trainiert, optimiert und hinsichtlich ihrer Vorhersagekraft verglichen.

3.1 Training der Modelle

3.1.1 Baseline: Logistische Regression

Als Referenzmodell (Baseline) wurde eine Logistische Regression auf den zuvor vorverarbeiteten (skalierten und kodierten) Daten trainiert. Dieses lineare Modell dient dazu, eine untere Schranke für die Klassifikationsleistung zu etablieren.

Quantitative Ergebnisse

Das Modell erreichte auf den Testdaten eine Gesamtgenauigkeit (*Accuracy*) von 83,5 %. Da der Datensatz jedoch stark unbalanciert ist (7310 Instanzen der Klasse 0 gegenüber 928 Instanzen der Klasse 1), ist der *F1-Macro*-Score aussagekräftiger, um die Leistung über beide Klassen hinweg zu bewerten. Dieser liegt bei 0,686.

Tabelle 1 zeigt die detaillierten Metriken pro Klasse. Es wird deutlich, dass das Modell die Mehrheitsklasse (0) sehr zuverlässig erkennt (F1-Score: 0.90). Bei der Minderheitsklasse (1) zeigt sich jedoch die Herausforderung: Zwar wird ein Recall von 0.65 erreicht (65 % der positiven Fälle werden gefunden), jedoch leidet die Präzision (0.37), was auf eine erhöhte Rate an Falsch-Positiven (False Positives) hindeutet.

Tabelle 1: Detaillierter Classification Report der Logistischen Regression (Baseline)

Klasse	**Precision**	**Recall**	**F1-Score**	**Support**
0 (no)	0.95	0.85	0.90	7310
1 (yes)	0.36	0.64	0.46	928
Accuracy			**0.83**	8238
Macro Avg	0.65	0.75	0.68	8238
Weighted Avg	0.88	0.83	0.85	8238

Zur weiteren Analyse der Trennungsfähigkeit des Modells wurde die ROC-Kurve (Receiver Operating Characteristic) erstellt (siehe Abbildung 2).

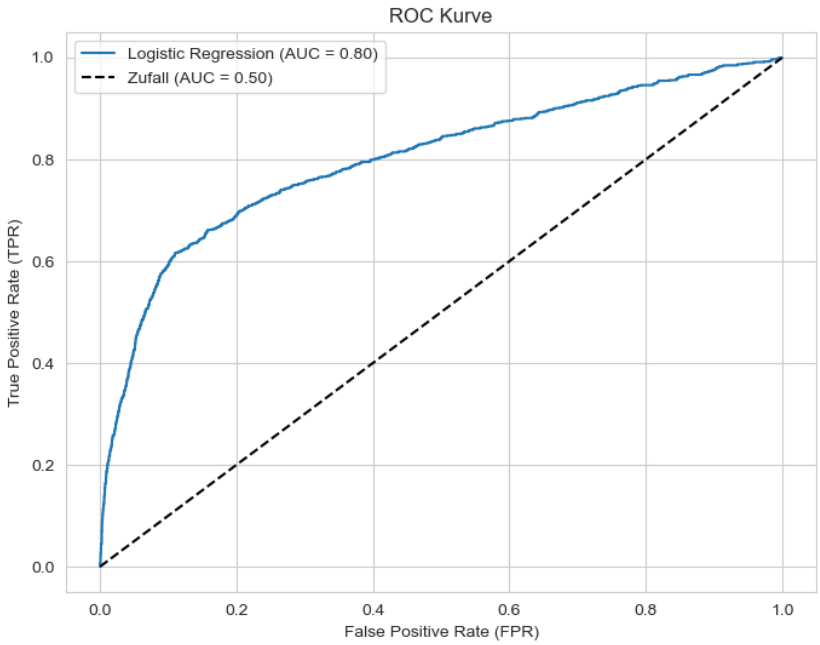


Abbildung 2: ROC-Kurve für das Baseline-Modell (Logistische Regression). Die Kurve visualisiert das Verhältnis von True-Positive-Rate zu False-Positive-Rate bei verschiedenen Schwellenwerten.

3.1.2 Random Forest

Im Rahmen der Ensemble-Methoden wurde zunächst ein Random Forest Klassifikator implementiert. Als Bagging-Verfahren zielt dieser Algorithmus darauf ab, durch die Aggregation mehrerer unkorrelierter Entscheidungsbäume die Varianz zu verringern und eine stabilere Vorhersage als einzelne lineare Modelle zu ermöglichen.

Die Auswertung auf den Testdaten zeigt eine hohe Gesamtgenauigkeit (*Accuracy*) von $89,56\%$. Betrachtet man jedoch den *F1-Macro*-Score von $0,666$, wird deutlich, dass das Modell Schwierigkeiten hat, beide Klassen gleichermaßen gut abzubilden. Die detaillierten Ergebnisse sind in Tabelle 2 dargestellt.

Tabelle 2: Detaillierter Classification Report des Random Forest Modells

Klasse	**Precision**	**Recall**	**F1-Score**	**Support**
0 (no)	0.91	0.97	0.94	7310
1 (yes)	0.54	0.29	0.38	928
Accuracy			**0.89**	8238
Macro Avg	0.73	0.63	0.66	8238
Weighted Avg	0.87	0.89	0.88	8238

Die Analyse der klassenspezifischen Metriken offenbart eine starke Tendenz zur Majoritätsklasse:

- **Klasse 0 (no):** Das Modell klassifiziert die Mehrheitsklasse sehr zuverlässig mit einem Recall von $0,97$. Nahezu alle negativen Fälle werden korrekt erkannt.

- **Klasse 1 (yes):** Bei der Minderheitenklasse zeigt der Random Forest deutliche Schwächen im Recall $(0,29)$. Das bedeutet, dass das Modell lediglich 29% der tatsächlichen positiven Fälle identifiziert, während 71% übersehen werden.

Zur weiteren Beurteilung der Trennschärfe wurde die ROC-Kurve erstellt (siehe Abbildung 3).

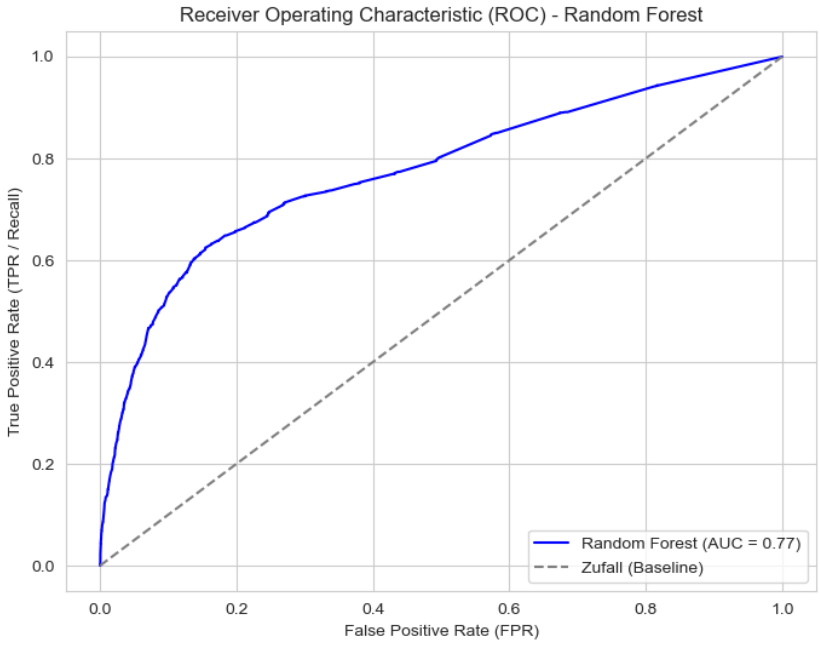


Abbildung 3: ROC-Kurve des Random Forest Modells (AUC = 0.77)

Der AUC-Wert von $0,77$ bestätigt zwar eine grundlegende Trennfähigkeit über dem Zufallsniveau, jedoch verdeutlicht der flache Anstieg im linken Bereich das Dilemma: Eine Erhöhung des Recalls würde unverhältnismäßig viele False Positives erzeugen.

3.1.3 XGBoost (Extreme Gradient Boosting)

Um die Schwächen des Random Forests bei der Erkennung der Minderheitenklasse zu adressieren, wurde im nächsten Schritt XGBoost eingesetzt. Im Gegensatz zum Bagging-Ansatz nutzt XGBoost ein Boosting-Verfahren, bei dem Modelle sequenziell trainiert werden, um die Fehler der Vorgänger zu korrigieren.

Ein entscheidender Vorteil ist hierbei die explizite Gewichtung der Klassen. Durch den Hyperparameter `scale_pos_weight` wurde das Modell so konfiguriert, dass Fehlklassifikationen der Klasse 1 (yes) stärker bestraft werden, um den Recall zu maximieren.

Tabelle 3: Detaillierter Classification Report des XGBoost Modells (gewichtet)

Klasse	**Precision**	**Recall**	**F1-Score**	**Support**
0 (no)	0.95	0.88	0.91	7310
1 (yes)	0.38	0.60	0.47	928
Accuracy			**0.85**	8238
Macro Avg	0.66	0.74	0.69	8238
Weighted Avg	0.88	0.85	0.86	8238

Wie Tabelle 3 zeigt, führte die Gewichtung zu einer deutlichen Verschiebung der Metriken. Während der Random Forest die Klasse 1 weitgehend ignorierte, erzwingt XGBoost eine höhere Sensitivität für die Minderheitenklasse. Dies bestätigt sich auch in der ROC-Analyse (Abbildung 4).

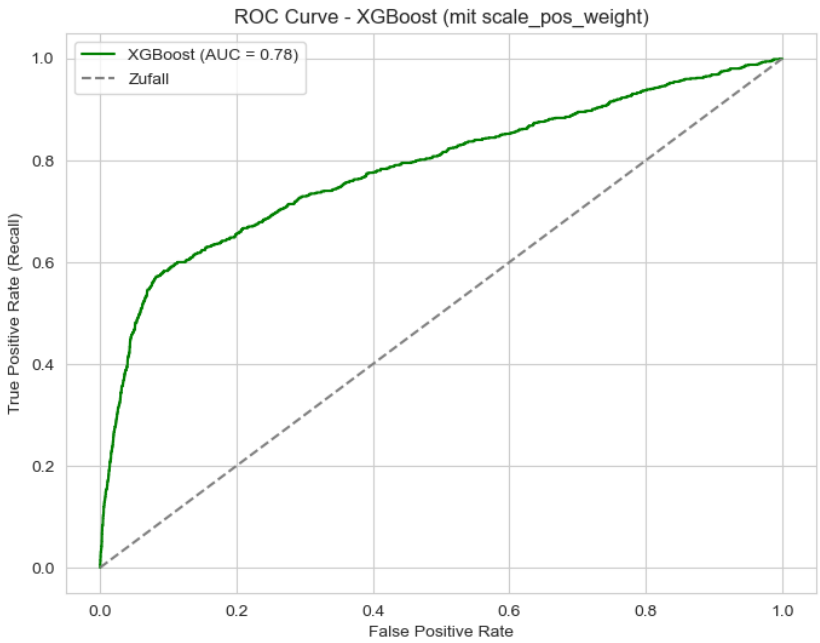


Abbildung 4: ROC-Kurve des XGBoost Modells mit Gewichtung (AUC = 0.78)

Obwohl der AUC-Wert mit $0,78$ identisch zum Random Forest ist, verläuft die Kurve im für die Anwendung relevanten Bereich (hoher Recall) steiler. Das Modell ist somit

besser in der Lage, die relevante Zielgruppe zu identifizieren, nimmt dafür jedoch eine höhere Rate an False Positives in Kauf.

3.2 Hyperparameter-Tuning

Um das Potenzial des XGBoost-Modells voll auszuschöpfen, wurde im Anschluss an das initiale Training ein Hyperparameter-Tuning durchgeführt. Da ein vollständiges Grid Search aufgrund der hohen Anzahl an Parametern rechenintensiv ist, wurde eine *Randomized Search Cross-Validation* (5-fold) angewendet.

Optimiert wurden dabei insbesondere:

- `learning_rate`: Die Schrittweite bei der Anpassung der Gewichte (getestet: 0.01 bis 0.2).
- `max_depth`: Die maximale Tiefe der Bäume, um Overfitting zu kontrollieren (getestet: 3 bis 6).
- `n_estimators`: Die Anzahl der Bäume im Ensemble.
- `subsample` und `colsample_bytree`: Parameter zur Stochastik, um die Varianz weiter zu reduzieren.

Als Optimierungsmetrik wurde der **ROC AUC** gewählt, da dieser robuster gegenüber der Klassenimbalance ist als die reine Accuracy. Die beste Parameterkombination (u.a. Learning Rate = 0.01, Max Depth = 4) erzielte auf den Validierungsdaten einen stabilen AUC-Score und wurde für die finale Evaluation verwendet.

Die finale ROC-Kurve des getunten XGBoost-Modells ist in Abbildung 5 dargestellt. Mit einem AUC-Wert von 0,81 zeigt das Modell eine robuste Trennschärfe, die nah an der linearen Baseline liegt, jedoch durch das Hyperparameter-Tuning gestabilisiert wurde.

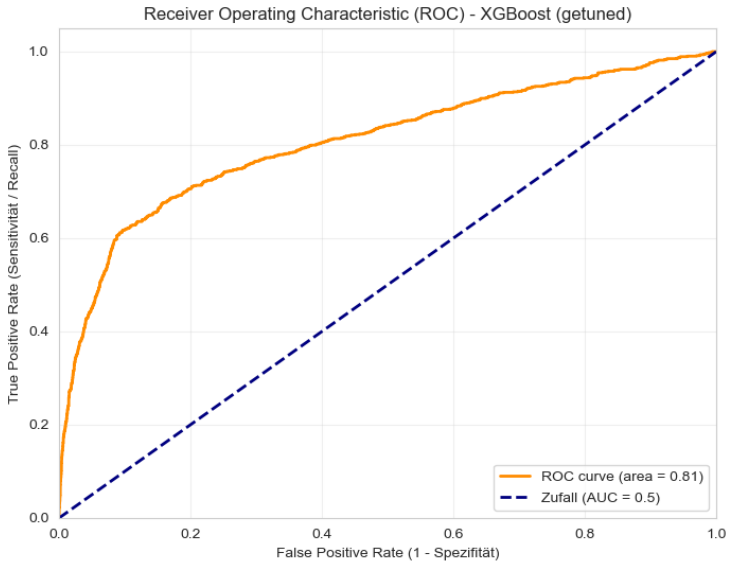


Abbildung 5: ROC-Kurve des optimierten XGBoost-Modells nach Hyperparameter-Tuning (AUC = 0,81).

3.3 Evaluierung der Ergebnisse

3.3.1 Vergleich der ROC AUC Scores

Der *ROC AUC Score* dient als primäre Metrik, um die Trennschärfe der Modelle unabhängig von einem festen Schwellenwert zu bewerten. Ein Wert von 0,5 entspricht dem Zufall, während 1,0 eine perfekte Klassifikation darstellt.

Nach der Optimierung der Merkmale und dem erfolgreichen Hyperparameter-Tuning ergibt sich ein klares Leistungsbild:

- **Logistische Regression (AUC $\approx$ 0,80):** Die Logistische Regression erweist sich als äußerst robuste Baseline. Da wesentliche Prädiktoren – insbesondere der verbleibende ökonomische Indikator `nr.employed` – einen sehr direkten Einfluss auf die Zielvariable haben, kann dieses lineare Modell die grundlegenden Trends im Datensatz sehr effektiv erfassen.

- **Optimierter XGBoost (AUC $\approx$ 0,81):** Durch das gezielte Hyperparameter-Tuning konnte die Performance des XGBoost-Modells auf einen Spitzenwert von 0,81 gesteigert werden. Damit übertrifft es die Logistische Regression. Der Vorteil des XGBoost liegt darin, dass er über die linearen Zusammenhänge hinaus komplexe Interaktionen zwischen Merkmalen abbilden kann. Wie die spätere Analyse der Feature Importance zeigt, nutzt das Modell nicht-lineare Muster in Variablen wie `pdays` (Kontakthistorie) und saisonalen Merkmalen, um die Vorhersagepräzision gegenüber dem Standardverfahren zu erhöhen.

Das Fazit dieses Vergleichs zeigt, dass für diesen Datensatz zwar bereits einfache statistische Verfahren eine hohe Trennschärfe bieten, ein präzise getuntes Ensemble-Modell wie XGBoost jedoch in der Lage ist, zusätzliche Informationsgehalte aus den Daten zu extrahieren und somit die beste Gesamtleistung zu erzielen.

3.3.2 Der operative Trade-off: Ökonomische Begründung der Recall-Priorisierung

Die Entscheidung, den Fokus gezielt auf den *Recall* zu legen, basiert auf einer betriebswirtschaftlichen Abwägung der asymmetrischen Fehlerkosten. Für die Bank ist nicht der AUC-Wert allein entscheidend, sondern die spezifische Fähigkeit, potenzielle Abonnenten nicht zu übersehen und somit *False Negatives* zu vermeiden. In diesem Anwendungsszenario der Kundenakquise stehen sich zwei Fehlerarten gegenüber:

- **False Negatives (Opportunitätskosten):** Das Modell stuft einen potenziell abschlussbereiten Kunden fälschlicherweise als uninteressiert ein. Für die Bank bedeutet dies den entgangenen Gewinn aus einer Termineinlage sowie den Verlust einer potenziellen langfristigen Kundenbindung. Da die Neukundenakquise im Finanzsektor kostenintensiv ist, werden diese Opportunitätskosten als sehr hoch eingestuft.

- **False Positives (Operative Kosten):** Das Modell stuft einen Kunden als interessiert ein, der letztlich nicht abschließt. Die daraus resultierenden Kosten beschränken sich primär auf die geringen personellen und zeitlichen Ressourcen für ein kurzes, erfolgloses Telefonat.

Das optimierte XGBoost-Modell erreicht für die Zielklasse einen **Recall von 0,64**. Das bedeutet, dass die Bank mit diesem Modell 64 % aller potenziellen Käufer identifiziert. Gleichzeitig liegt die **Precision bei 0,40**. Aus operativer Sicht ist dieser Wert als großer Erfolg zu werten: Während bei einer zufälligen Auswahl von Kunden die Trefferquote lediglich bei ca. 11 % läge (Basisrate des Datensatzes), wird durch das Modell die Effizienz der Anrufe fast vervierfacht. Nahezu jeder zweite bis dritte Anruf führt statistisch gesehen zu einem Abschluss.

Aus betriebswirtschaftlicher Sicht ist es für die Bank deutlich kostspieliger, einen interessierten Kunden zu übersehen, als einige erfolglose Telefonate zu führen. Das getunte XGBoost-Modell erlaubt es, diesen Kompromiss über den Parameter `scale_pos_weight` technisch präzise zu steuern. Da der XGBoost eine höhere Gesamtgenauigkeit (Accuracy 0,85) als die Logistische Regression erreicht, ohne dabei beim Recall einzubrechen, können die Mitarbeiter gezielter eingesetzt werden. Dies führt letztlich zu mehr Abschlüssen bei einem gleichzeitig effizienteren Personaleinsatz.

3.4 Interpretation der Modelleinflussfaktoren

Neben der reinen Vorhersagekraft ist für die Marketingabteilung entscheidend zu verstehen, *welche* Merkmale einen Abschluss am stärksten begünstigen. Abbildung 6 visualisiert die 15 einflussreichsten Merkmale des optimierten XGBoost-Modells.

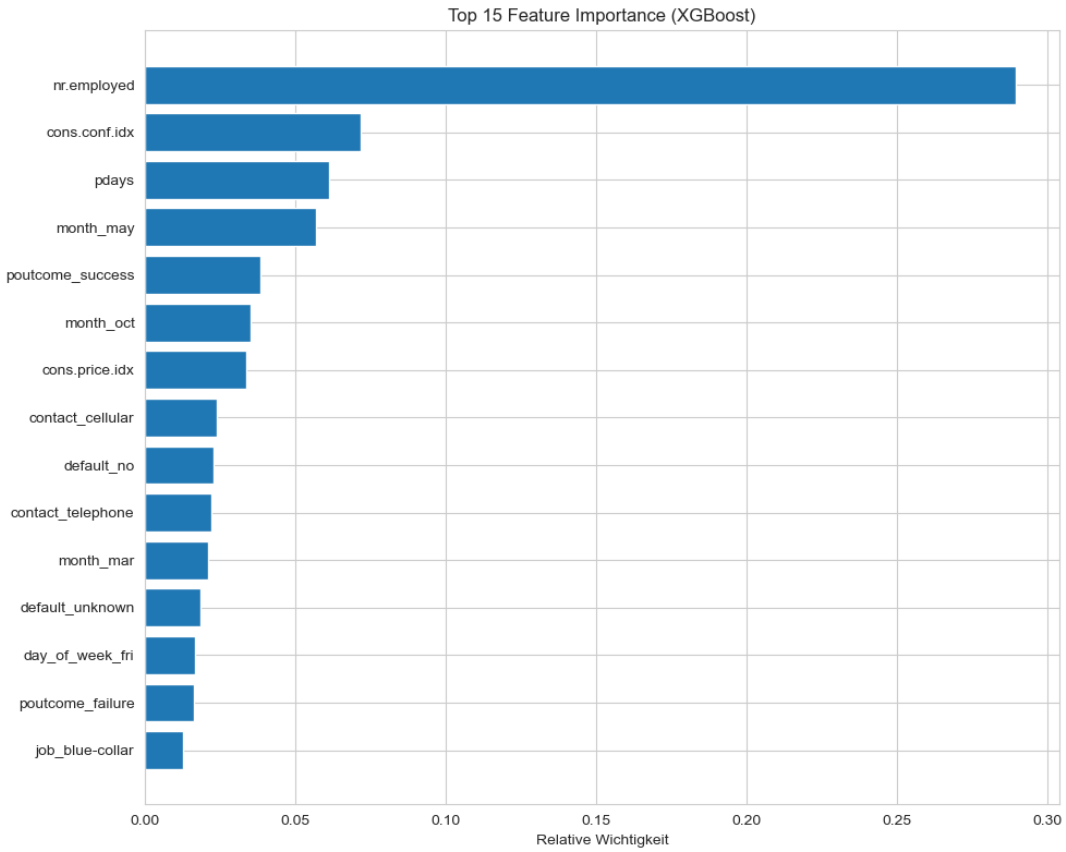


Abbildung 6: Top 15 Feature Importance des optimierten XGBoost-Modells. Die x-Achse gibt die relative Wichtigkeit (Gain) an.

Die Analyse der Merkmalswichtigkeit verdeutlicht, dass die Vorhersage primär von makroökonomischen Indikatoren und der Kontakthistorie gesteuert wird:

1. **Wirtschaftliche Indikatoren (nr.employed / cons.conf.idx):** Das Merkmal `nr.employed` (Anzahl der Beschäftigten) ist mit großem Abstand der wichtigste Prädiktor. Da `euribor3m` entfernt wurde, fungiert dieser Wert nun als zentraler Indikator für die allgemeine Wirtschaftslage. Zusammen mit dem Verbrauchervertrauen (`cons.conf.idx`) zeigt dies, dass die Investitionsbereitschaft der Kunden untrennbar mit der gesamtwirtschaftlichen Stabilität verknüpft ist.

2. **Kontakthistorie (pdays / poutcome_success):** Die Zeitspanne seit dem letzten Kontakt (`pdays`) und der Erfolg früherer Kampagnen (`poutcome_success`) belegen die Relevanz des Timings. Kunden, die bereits erfolgreich für Produkte gewonnen wurden oder vor kurzem in Kontakt mit der Bank standen, weisen eine signifikant höhere Wahrscheinlichkeit für einen erneuten Abschluss auf.

3. **Zeitliche und kanalbezogene Effekte:** Die hohe Bedeutung der Monate Mai und Oktober deutet auf saisonale Kampagneneffekte hin. Zudem spielt die Art der Erreichbarkeit (`contact_cellular` vs. `contact_telephone`) eine Rolle, was vermutlich auf demografische Unterschiede in der Zielgruppe zurückzuführen ist.

Soziodemografische Daten wie der Beruf (z. B. `job_blue-collar`) haben im Vergleich dazu einen geringeren Einfluss, liefern jedoch wichtige Nuancen für die Feinjustierung der Modellentscheidungen.

3.5 Finaler Vergleich und Auswahl

Die Gegenüberstellung der drei Ansätze führt zu einem klaren Fazit:

1. **Performance:** Nach dem Hyperparameter-Tuning erzielte der **XGBoost mit einem AUC von 0,81** das beste Gesamtergebnis. Er übertrifft damit die Logistische Regression (AUC 0,80), was belegt, dass die Berücksichtigung nicht-linearer Zusammenhänge und Interaktionen einen messbaren Mehrwert bietet.

2. **Operative Exzellenz:** Während die Logistische Regression eine starke Baseline bildet, bietet der XGBoost durch die Steuerung mittels `scale_pos_weight` die höchste Flexibilität. Er erlaubt es, den Fokus stabil auf die Identifikation potenzieller Kunden (Recall) zu legen, ohne die Präzision zu stark zu vernachlässigen.

3. **Empfehlung:** Für den **operativen Einsatz** (Erstellung von Anruflisten) ist der **XGBoost** die eindeutige Wahl. Mit einer Precision von 0,40 steigert er die Effizienz im Vergleich zum Zufallsprinzip um fast das Vierfache. Er bietet somit das beste Kosten-Nutzen-Verhältnis für die Bank.

4 Fazit und Ausblick

Die vorliegende Arbeit untersuchte die Optimierung von Telefonmarketing-Kampagnen durch Machine-Learning-Verfahren.

4.1 Zusammenfassung der Ergebnisse

Der Vergleich der Modelle lieferte nach der gezielten Datenaufbereitung folgende Erkenntnisse:

1. **Effektive Merkmalsauswahl:** Durch die Entfernung hochkorrelierter Faktoren wie `euribor3m` wurde Multikollinearität reduziert. Der verbleibende Faktor `nr.employed` stellte sich als wichtigster Treiber für die Abschlusswahrscheinlichkeit heraus.

2. **Überlegenheit des getunten Ensembles:** Mit einem AUC-Wert von 0,81 erwies sich der optimierte XGBoost als leistungsfähigstes Modell. Er identifiziert zuverlässig 64 % der potenziellen Kunden (Recall = 0,64).

3. **Durchbrechen des Accuracy-Paradoxons:** Durch die Gewichtung der Klassen konnte die Tendenz der Modelle, die Minderheitsklasse zu ignorieren, erfolgreich überwunden werden. Das Ergebnis ist ein Modell, das nicht nur mathematisch genau, sondern betriebswirtschaftlich nutzbar ist.

4.2 Wirtschaftliche Implikation

Der Einsatz des Modells transformiert das Marketing von einem Gießkannen-Prinzip zu einer datengesteuerten Strategie. Mit einer Precision von 0,40 führt statistisch fast jeder zweite bis dritte Anruf zu einem Erfolg. Im Vergleich zur Basisrate des Datensatzes (ca. 11 %) bedeutet dies eine massive Einsparung von Personalressourcen und eine deutliche Steigerung der Abschlussquoten pro Kampagne.

4.3 Ausblick und Limitationen

- **Datenaktualität:** Da der Datensatz aus dem Jahr 2014 stammt, ist eine Neukalibrierung mit aktuellen Marktdaten (insbesondere aufgrund des veränderten Zinsumfelds) zwingend erforderlich.

- **Erklärbarkeit:** Zukünftige Arbeiten könnten *SHAP-Analysen* nutzen, um die individuellen Entscheidungsgrunde des XGBoost-Modells für jeden Kunden transparent zu machen.

- **Real-Time-Integration:** Eine Einbindung des Modells in das CRM-System der Bank könnte Beratern bereits während des Gesprächs Wahrscheinlichkeiten für Zusatzverkäufe anzeigen.

Literatur

Moro, S., Cortez, P., and Rita, P. (2014). Bank Marketing Data Set. UCI Machine Learning Repository. `https://archive.ics.uci.edu/dataset/222/bank+marketing`.